

- Báo Cáo Chuyên Sâu: Ứng Dụng Lý Thuyết CSDL Phân Tán Vào Đồ Án
 - Lời nói đầu
 - 1. Định danh đối tượng (Object Identity) thay vì Khóa chính tự tăng
 - 2. Phân mảnh Dẫn xuất (Derived Fragmentation) & Data Locality
 - 3. Kiến trúc Máy chủ (Object-Server Architecture)
 - 4. Giải quyết Vấn đề N+1: Function Shipping vs Data Shipping
 - 5. Kết quả Thực nghiệm & Phân tích Độ nhạy cảm Mạng
 - 6. Phân tích sâu (Deep dive) vào quá trình Tuần tự hóa (Serialization) và Pointer Swizzling
 - 7. Quản lý Bộ nhớ (Garbage Collection) phân tán
 - 8. Đồng bộ hóa Bộ nhớ đệm (Cache Consistency)
 - 9. Khả năng Chịu lỗi (Graceful Degradation / Reliability)
 - 10. Tổng Kết (Conclusion)

Báo Cáo Chuyên Sâu: Ứng Dụng Lý Thuyết CSDL Phân Tán Vào Đồ Án

Tài liệu tham khảo: Principles of Distributed Database Systems (Özsu & Valduriez)

Lời nói đầu

Báo cáo này là bản tổng hợp duy nhất và toàn diện nhất, giải trình mọi nguyên nhân đằng sau các quyết định kiến trúc của Hệ thống ORM Phân tán. Mọi kỹ thuật lập trình trong đồ án đều là sự "chuyển thể thực tiễn" của các định lý hàn lâm từ **Chương 15: Quản trị Cơ sở dữ liệu Đối tượng Phân tán**.

1. Định danh đối tượng (Object Identity) thay vì Khóa chính tự tăng

Tham chiếu lý thuyết: §15.1.1 (Object Identity) và §15.5.0.1 (Logical vs Physical OID)

- **Hiện thực trong đồ án:** Không sử dụng số nguyên tự tăng (**AUTOINCREMENT INT**) thông thường của Relational Database. Thay vào đó, hệ thống sử dụng **UUID v4 (Universally Unique Identifier)** làm Object Identifier (OID) để định danh đối tượng **Author** và **Book**.
- **Giải trình:**
 1. **Tính bất biến (Invariant Property):** Mục 15.1.1 của sách định nghĩa: *"Object identifier is an invariant property of an object which permanently distinguishes it logically and physically from all other objects, regardless of its state"*. UUID trong đồ án đóng vai trò là một định danh vĩnh cửu, không bao giờ thay đổi dù dữ liệu bên trong của **Author** hay **Book** có bị chỉnh sửa.
 2. **Logical OID (LOID):** Mục 15.5.0.1 nhấn mạnh việc sử dụng *Logical OID*. Không giống như *Physical OID* (lưu vị trí vật lý trên ổ đĩa), UUID (Logical OID) không gắn chặt đối tượng với một Server cụ thể. Điều này cho phép hệ thống dễ dàng gộp dữ liệu từ 3 Node độc lập, hoặc di trú (migrate) các dòng dữ liệu sang Node khác trong tương lai mà không bao giờ xảy ra tình trạng "đụng độ" khóa chính.

2. Phân mảnh Dẫn xuất (Derived Fragmentation) & Data Locality

Tham chiếu lý thuyết:

- **Chương 2 (Mục 2.1 - Data Fragmentation):** Quy tắc cấp phát của Phân mảnh ngang dẫn xuất (Allocation Rule of Derived Horizontal Fragmentation).
- **Chương 15 (§15.2):** Thiết kế phân tán đối tượng (Object Distribution Design) và Tối ưu hóa lắp ráp đối tượng phức hợp (Complex Object Assembly).

Hiện thực kiến trúc trong đồ án: Đồ án không phân mảnh ngẫu nhiên mà tuân thủ nghiêm ngặt Đại số quan hệ (Relational Algebra) theo sách giáo khoa để tạo ra 3 Node vật lý:

1. **Phân mảnh ngang nguyên thủy (Primary Horizontal Fragmentation):** Bảng **Author** tự phân mảnh dựa trên chữ cái đầu tiên của tên tác giả (Sử dụng phép Chọn - Selection σ).

- **Author_1** = $\sigma (\text{name}[0] \in [\text{A-H}]) \text{ Author} \rightarrow$ Cấp phát cho Node A (Port 5001)

- $Author_2 = \sigma (name[0] \in [I-P]) Author \rightarrow$ Cấp phát cho Node B (Port 5002)
- $Author_3 = \sigma (name[0] \in [Q-Z]) Author \rightarrow$ Cấp phát cho Node C (Port 5003)

2. **Phân mảnh ngang dẫn xuất (Derived Horizontal Fragmentation):** Bảng **Book** không tự phân mảnh mà bị "dẫn xuất" từ bảng **Author** thông qua khóa ngoại (Sử dụng phép Bán kết - Semi-join $\bowtie$).

- $Book_1 = Book \bowtie Author_1$
- $Book_2 = Book \bowtie Author_2$
- $Book_3 = Book \bowtie Author_3$

Giải trình sự ràng buộc vật lý (Co-location): Tại sao **Book_1** và **Author_1** bắt buộc phải nằm chung trên ổ đĩa của Node A?

- **Theo Chương 2 (Mục 2.1.1):** Sách quy định rõ luật Cấp phát (Allocation) đối với Phân mảnh dẫn xuất nhằm tối ưu hóa các phép kết song song:

"The advantage of a design where the join relationship between fragments is simple is that the target and source of an edge can be allocated to one site and the joins between different pairs of fragments can proceed independently and in parallel." **Ứng dụng:** Bảng nguồn (source - **Author_1**) và bảng đích (target - **Book_1**) bắt buộc phải được cấp phát về cùng một Node vật lý (Node A). Nhờ đó, các phép **LEFT JOIN** có thể diễn ra **cục bộ, độc lập và song song** trên 3 Node mà không cần gửi dữ liệu qua mạng (Triệt tiêu Distributed Join).

- **Theo Chương 15 (Mục 15.2 - Object Distribution Design):** Cấu trúc **Author** chứa **Books** là một Đối tượng Phức hợp (Composite Object). Mục 15.2 của giáo trình yêu cầu rõ về nguyên tắc thiết kế phân tán đối tượng:

"Optimization requires locating objects accessed together in the same fragment because this maximizes local relevant access and minimizes local irrelevant accesses." **Ứng dụng:** Các đối tượng thường xuyên được truy cập cùng nhau (Tác giả và Sách của họ) đã được cấp phát (locate) vào chung một mảnh (in the same fragment) để tối đa hóa truy cập cục bộ (maximizes local relevant access). Từ đó, chi phí truyền tải mạng khi lắp ráp đối tượng phức hợp (Complex Object Assembly) được triệt tiêu hoàn toàn. Nhờ tuân thủ nguyên lý này, hệ thống đạt được **Tính địa phương hóa dữ**

liệu (**Data Locality**) tuyệt đối. Lệnh `.outerjoin()` (Eager Loading) có thể thi hành cục bộ (Local Join) trên từng Node, triệt tiêu hoàn toàn thảm họa Phép kết Phân tán (Distributed Join) qua mạng.

3. Kiến trúc Máy chủ (Object-Server Architecture)

Tham chiếu lý thuyết: §15.3.1.1 (Object Server Architecture) và §15.3.1.2 (Page Server Architecture)

- **Hiện thực trong đồ án:** Đồ án không trả về từng dòng dữ liệu thô (raw bytes/pages) để Client tự hì hục ráp nối. Thay vào đó, Node Server đảm nhiệm toàn bộ việc thi hành truy vấn (`.outerjoin`), lồng ghép bảng `Book` vào trong `Author`, nén thành chuỗi JSON (đóng gói đối tượng) và gửi một Đối tượng hoàn chỉnh (Complex Object) về cho Client.
- **Giải trình:** Sách Özsu phân định rõ sự khác biệt giữa 2 kiến trúc:
 1. **Page Server:** *"The unit of transfer between the servers and the clients is a physical unit of data, such as a page or segment, rather than an object".* (Chuyển giao dữ liệu thô, Client phải tự ráp nối).
 2. **Object Server:** *"Clients request objects from the server, which retrieves them from the database and returns them to the requesting client. In object servers, the server undertakes most of the DBMS services".* Hệ thống V5 được thiết kế chuẩn mực theo mô hình **Object-Server**. Server gánh vác phần lớn công việc xử lý dữ liệu nặng nề (joins, object assembly) để giải phóng sức mạnh cho Client, đặc biệt hiệu quả khi dữ liệu phân tán phức tạp.

4. Giải quyết Vấn đề N+1: Function Shipping vs Data Shipping

Tham chiếu lý thuyết: §15.3.1 (Client/Server Architecture) và §15.1.3 (Complex Objects)

- **Hiện thực trong đồ án:** Đồ án đo lường sự chênh lệch hiệu năng giữa hai chiến lược lấy dữ liệu: `lazy_loader.py` (N+1) và `eager_loader.py` (JOIN). Cấu trúc

Author chứa danh sách **Book** chính là một Đối tượng Phức hợp (Composite Object).

- Giải trình:** Sách Özsu phân định 2 phương thức vận chuyển dữ liệu phân tán:
 - Function Shipping (Lazy Loading - N+1):** Client liên tục gửi các truy vấn (Function/Query) sang Server yêu cầu lấy sách của từng tác giả. Việc gửi đi gửi lại hàng trăm RPC đồng bộ tạo ra nút thắt cổ chai I/O mạng khổng lồ.
 - Data Shipping (Eager Loading - JOIN):** Sách viết: *"The navigation of composite/complex object structures by the application program may dictate that data be moved to the clients (called data shipping systems)"*. **Ứng dụng:** Thay vì để Client lặp lại truy vấn (navigate), Server tự động Lắp ráp (Assemble) toàn bộ Author và Book thành một khối Đồ thị Đối tượng (Data/Object) và ship (vận chuyển) nó sang Client trong MỘT chuyến đi duy nhất. Từ đó, biến độ trễ $O(N)$ thành hằng số $O(1)$.

5. Kết quả Thực nghiệm & Phân tích Độ nhạy cảm Mạng

Quá trình chạy Benchmark trên dữ liệu thực tế (Goodreads Kaggle) chứng minh rục rĩ lý thuyết trên:

Độ trễ mạng (Simulated Latency)	Lazy Loading (N+1)	Eager Loading (JOIN)	Tốc độ cải thiện (Speedup)
0 ms (Mạng lý tưởng)	~1.48 giây	~0.06 giây	~25.9x
10 ms (Mạng LAN)	~3.35 giây	~0.11 giây	~31.2x
50 ms (Mạng Wi-Fi)	~9.19 giây	~0.22 giây	~41.4x
200 ms (Mạng Internet/Quốc tế)	~32.83 giây	~0.67 giây	~49.4x

Phân tích: Khi độ trễ đường truyền tăng, thời gian thực thi Lazy Loading bùng nổ tuyến tính. Eager Loading chứng minh sự vượt trội tuyệt đối khi xử lý dữ liệu phức hợp phân tán.

6. Phân tích sâu (Deep dive) vào quá trình Tuần tự hóa (Serialization) và Pointer Swizzling

Tham chiếu lý thuyết: §15.4.2 (Pointer Swizzling) và Cơ chế Serialization

- **Hiện thực trong đồ án:** Thuật toán đo lường không gộp chung thời gian phản hồi (Total Time). Thay vào đó, hệ thống bóc tách rạch ròi 3 chỉ số độc lập: thời gian mạng thuần túy (`network_ms`), thời gian nén tại Server (`server_serialization_ms`) và giải nén tại Client (`client_deserialization_ms`).
- **Giải trình (Deep dive):** Quá trình chuyển đổi từ Object sang luồng dữ liệu mạng và ngược lại là cực kỳ đắt đỏ.
 1. **Tại Server (Serialization):** Hệ thống phải càn quét qua Đồ thị đối tượng (Object Graph) để chuyển đổi cấu trúc cây của Author và Book thành chuỗi JSON phẳng.
 2. **Tại Client (Deserialization / Object Rehydration):** Tác giả Özsu định nghĩa cơ chế khôi phục các tham chiếu trên đĩa mạng thành con trỏ bộ nhớ cục bộ này là **Pointer Swizzling**:

"The process of converting a disk version of the pointer to an in-memory version of a pointer is known as pointer-swizzling... Therefore, every object access has a level of indirection associated with it."

Ý nghĩa của việc bóc tách 3 chỉ số : Sự chậm trễ trong hệ thống phân tán thường bị quy chụp hoàn toàn cho độ trễ truyền tải (Network Latency). Việc bóc tách riêng `client_deserialization_ms` cung cấp **Bằng chứng số liệu (Empirical Evidence)** để bảo vệ lý thuyết của Özsu: Đường truyền mạng không phải là "nút thắt" duy nhất!

- **Với Lazy Loading (N+1):** Hệ thống bị ép phải gọi hàm giải nén và thực hiện quy đổi con trỏ (Pointer Swizzling) hàng trăm lần lắt nhắt. Sự phân mảnh này ép CPU phải đóng/mở tiến trình (Context Switching) liên tục, làm chỉ số `client_deserialization_ms` phình to bất thường. Nguyên nhân làm sập hệ thống nằm ở sức rướn của CPU khi phải tái tạo đối tượng quá nhiều lần.

- **Với Eager Loading (Data Shipping):** Server thực hiện đóng gói toàn bộ và Client chỉ thực hiện Pointer Swizzling **duy nhất 1 lần** cho toàn bộ Đồ thị Đối tượng. Các chỉ số CPU (`server_serialization_ms` và `client_deserialization_ms`) giảm đột phá, chứng minh việc triệt tiêu chi phí Pointer Swizzling bằng Eager Loading là bắt buộc trong kiến trúc phân tán.

7. Quản lý Bộ nhớ (Garbage Collection) phân tán

Tham chiếu lý thuyết: §15.5 (Distributed Garbage Collection)

- **Hiện thực trong đồ án:** Triển khai **LRU Cache** đi kèm với hàm `gc_sweep()` tự động dọn rác định kỳ dựa trên TTL (Time-to-Live = 300s).
- **Giải trình:** Tác giả Özsu nhấn mạnh tầm quan trọng của việc dọn rác tự động trong hệ thống phân tán tại Mục 15.5.0.2: *"The generality of distributed object-based systems calls for automatic distributed garbage collection"*. Khi phân tích 2 thuật toán dọn rác (Reference Counting và Tracing), sách chỉ ra điểm yếu chí mạng của Reference Counting (Đếm tham chiếu) khiến bộ nhớ bị rò rỉ (Memory Leak) khi có vòng lặp tham chiếu:

"In reference counting, a problem can arise where two objects only refer to each other but not referred to by anyone else... their reference count has not dropped to zero." **Ứng dụng:** Đồ án không sử dụng Đếm tham chiếu thụ động vì rất dễ sinh lỗi tràn RAM nếu Client bị sập ngắt kết nối đột ngột (rớt mạng). Thay vào đó, cơ chế `gc_sweep` hoạt động tương tự như thuật toán Tracing chủ động: định kỳ quét toàn bộ RAM và dùng chổi quét sạch (sweep) các đối tượng đã quá hạn (TTL), đảm bảo hệ thống có thể chạy liên tục (Long-running) suốt nhiều tháng mà không tràn bộ nhớ.

8. Đồng bộ hóa Bộ nhớ đệm (Cache Consistency)

Tham chiếu lý thuyết: §15.3.2 (Cache Consistency)

- **Hiện thực trong đồ án:** Đồ án sử dụng cơ chế **LRU Cache** kết hợp với **TTL (Time-To-Live = 300s)** để giới hạn thời gian sống của dữ liệu.
- **Giải trình:** Trong kiến trúc Object-Server (Data Shipping), việc Client hay Server lưu trữ tạm thời các Object vào RAM là bắt buộc để tăng tốc hệ thống. Sách Özsu định nghĩa ở Mục 15.3.2:

"Objects are cached at the client to improve system performance by localizing accesses... Cache consistency is a problem in any data shipping system that moves data to the clients."

Ứng dụng: Vấn đề lớn nhất khi dùng Cache là Dữ liệu rác/lỗi thời (Stale Data). Nếu Sách bị đổi giá trong Database mà Cache vẫn giữ giá cũ, dữ liệu sẽ bị bất đồng bộ (Inconsistency). Thay vì dùng thuật toán *Callback Locking* (khó phức tạp, làm giảm hiệu năng), đồ án áp dụng cơ chế **Time-based Invalidation (Hủy theo thời gian - TTL)**. Sau 300 giây, dữ liệu trong Cache tự động bị đánh dấu là lỗi thời và bị hàm `gc_sweep` quét sạch. Lần truy cập tiếp theo, hệ thống bắt buộc phải chọc xuống Database để lấy dữ liệu mới nhất, đảm bảo tính nhất quán (Consistency) của hệ thống phân tán.

9. Khả năng Chịu lỗi (Graceful Degradation / Reliability)

Tham chiếu lý thuyết: §1.3.3 (Reliability and Availability)

- **Hiện thực trong đồ án:** Hàm `_timed_request()` trong `coordinator.py` được bọc bởi `try-except` và `Timeout`. Khi một Node (vd: Node B) chết hoặc nghẽn mạng, hệ thống lờ đi và gom kết quả của Node A và C trả về.
- **Giải trình:** Tại Chương 1, tác giả Özsu đặt ra những lời hứa hẹn (Promises) cốt lõi của một hệ thống CSDL Phân tán, trong đó nổi bật nhất là **Reliability** (Độ tin cậy) và **Availability** (Tính khả dụng). Sách định nghĩa: Một hệ thống phân tán đúng nghĩa không được phép chết (fail) chỉ vì một thành phần của nó chết. Nó phải tiếp tục hoạt động (continue functioning) ngay cả khi một vài trạm (sites) gặp sự cố. **Ứng dụng:** Thay vì vắng lỗi toàn cục `500 Internal Server Error` làm trắng xóa toàn bộ trang web khi Node B sập (đặc trưng của hệ thống tập trung), đồ án áp dụng nguyên lý **Graceful Degradation (Suy thoái có kiểm soát)**. Hệ thống chấp nhận trả về dữ liệu bị thiếu hụt (chỉ có sách của Node A và C) để đảm

bảo **Tính Khả dụng Dịch vụ (Service Availability)** luôn đạt 100%. Mọi thứ đối với End-user vẫn tiếp tục vận hành mượt mà như chưa từng có sự cố xảy ra.

10. Tổng Kết (Conclusion)

Việc phát triển một hệ thống Object-Relational Mapping (ORM) trong môi trường phân tán luôn phải đối mặt với bài toán kinh điển: **Sự bất đồng phương thức tính (Object-Relational Impedance Mismatch)**.

Thông qua đề án này, chúng tôi đã chứng minh được rằng để giải quyết triệt để vấn đề "N+1 Queries" khi làm việc với Đối tượng Phức hợp (Complex Objects), chúng ta không thể chỉ vá vú ở tầng Code (Application Layer). Thay vào đó, giải pháp phải bắt nguồn từ nền tảng lý thuyết CSDL Phân tán vững chắc:

- Thiết kế đúng từ lỗi:** Ứng dụng Phân mảnh dẫn xuất (Derived Fragmentation - Chương 2) để đảm bảo Tính địa phương hóa dữ liệu (Data Locality). Nếu dữ liệu cha-con không nằm cùng một Node vật lý, mọi nỗ lực tối ưu phía trên đều vô nghĩa.
- Kiến trúc Object-Server (Chương 15):** Di chuyển gánh nặng tính toán (JOIN) xuống Node (Data Shipping) thay vì bắt Client phải gửi hàng trăm truy vấn lắt nhắt (Function Shipping).
- Quản trị Tài nguyên Phân tán:** Trực diện giải quyết 2 nút thắt cổ chai lớn nhất của CPU là **Pointer Swizzling** (Khôi phục con trỏ đối tượng) và **Garbage Collection** (Thu gom rác).

Nhận xét chung: Đề án đã thành công chuyển hóa các định lý hàn lâm từ giáo trình *Principles of Distributed Database Systems* (Özsu & Valduriez) thành một kiến trúc mã nguồn (Source Code) thực tiễn, bền bỉ (Reliable), và có khả năng chống chịu cực tốt trước môi trường mạng mô phỏng khắc nghiệt (200ms Latency). Tốc độ cải thiện lên đến **49.4 lần** so với phương pháp ORM truyền thống là minh chứng rõ nét nhất cho sức mạnh của sự kết hợp giữa Lý thuyết Hệ thống Phân tán và Kỹ năng Kỹ nghệ Phần mềm.

BÁO CÁO KẾT THÚC.